State Machines, Multi-Agent Topologies, and Production Reliability

Engineering resilient, cyclic, and observable agent runtimes with state persistence and human oversight.

1. State & Reducers

Deterministic Core

- TypedDict & Pydantic state
- Append-only message reducers
- Memory, Postgres, Redis savers

`Time-Travel Checkpointing`

2. Tool Sandboxing

Execution Isolation

- Function calling schemas
- E2B microVMs & Docker sandboxes
- 4-tier error reflection & retries

`Host Process Defense`

3. Multi-Agent Topologies

Context Boundaries

- Routers, Supervisors, Swarms
- Ephemeral child threads
- Token blowup prevention

`Decoupled Sub-Graphs`

4. HITL & Production SLAs

Production Governance

- Interrupt before side-effects
- State mutation on resume
- OTel distributed tracing

`Zero-Leak Guardrails`

ORCHESTRATION ECOSYSTEM

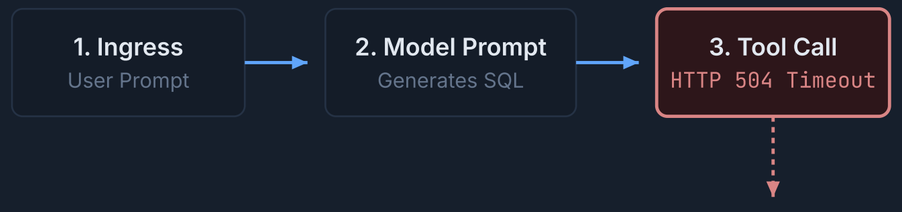
LangGraph · LlamaIndex Workflows · AutoGen (MSFT) · CrewAI · Semantic Kernel · OpenAI Swarm

SUPPORTING INFRASTRUCTURE: PostgreSQL (pgvector) · Redis Checkpointer · Docker / E2B Sandbox · OpenTelemetry / LangSmith

Why Linear DAGs Fail in Production AI Workloads

Directed Acyclic Graphs assume forward-only execution; real-world tool failures and model hallucinations demand cycles.

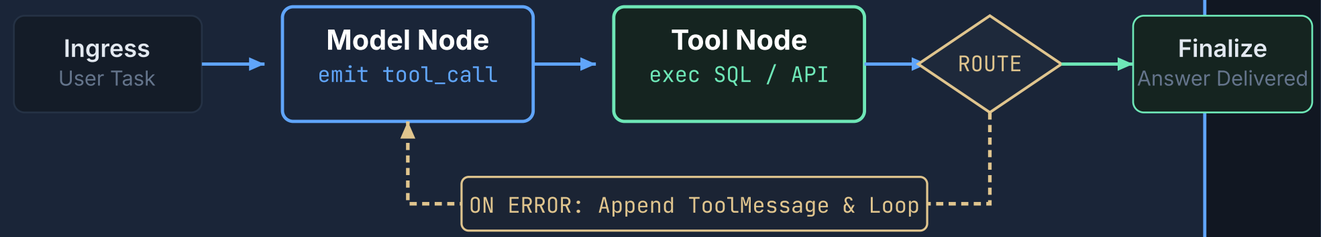
Linear DAG: The Fragile Forward Pipeline



ARCHITECTURAL FLAWS OF DAG RUNTIMES

- Cannot represent while-loops or iterative query refinement
- Tool failures bubble up directly to client applications
- Rigid graph structures cannot branch conditionally on runtime values

Cyclic State Machine: Enterprise Self-Correction



CYCLIC STATE ADVANTAGES

DETERMINISTIC RETRY GOVERNOR

State retains retry_counter: int. Graph enforces hard cap (max 3 cycles) before routing to a deterministic fallback handler.

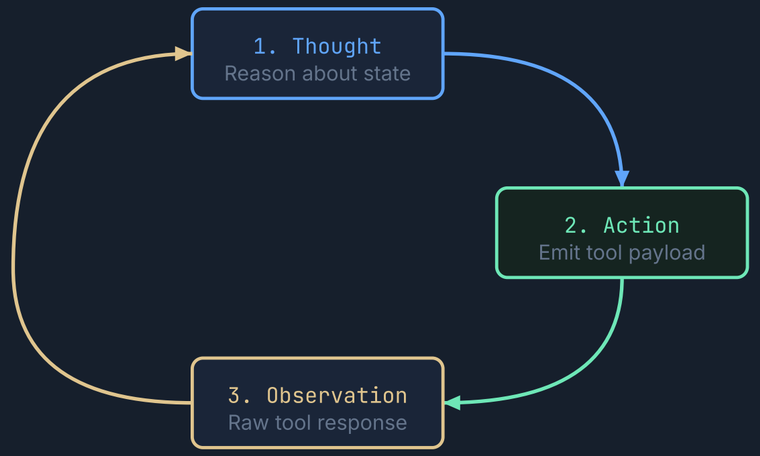
MODEL SELF-REFLECTION

Error details (e.g., invalid column name) are fed directly into the model, allowing it to adjust arguments rather than failing the user request.

The ReAct Loop: Fragility at Enterprise Scale

While ReAct enables dynamic tool usage, unconstrained prompt-based loops introduce severe operational vulnerabilities.

ReAct Cycle: Compounding Token Consumption



QUADRATIC TOKEN ACCUMULATION PER ROUNDTRIP:

Turn 1: 1.2k tokens → Turn 2: 3.8k tokens → Turn 3: 8.5k tokens → Turn 4: 1

The Four Production Failure Modes of Raw ReAct

1. CONTEXT WINDOW POISONING & NEEDLE DRIFT

Every raw tool return (e.g. 50-row SQL dumps) stays in memory. Attention on initial system prompt degrades as sequence length grows, inducing semantic drift and loss of original user constraints.

2. INFINITE OSCILLATION & TOOL RETRY LOOPS

When a tool returns an error, the model repeatedly alternates between two identical failing tools without adjusting parameters. Raw ReAct lacks stateful cycle counters to break infinite loops.

3. LATENCY SLA BLOWUP (p99 > 45 SECONDS)

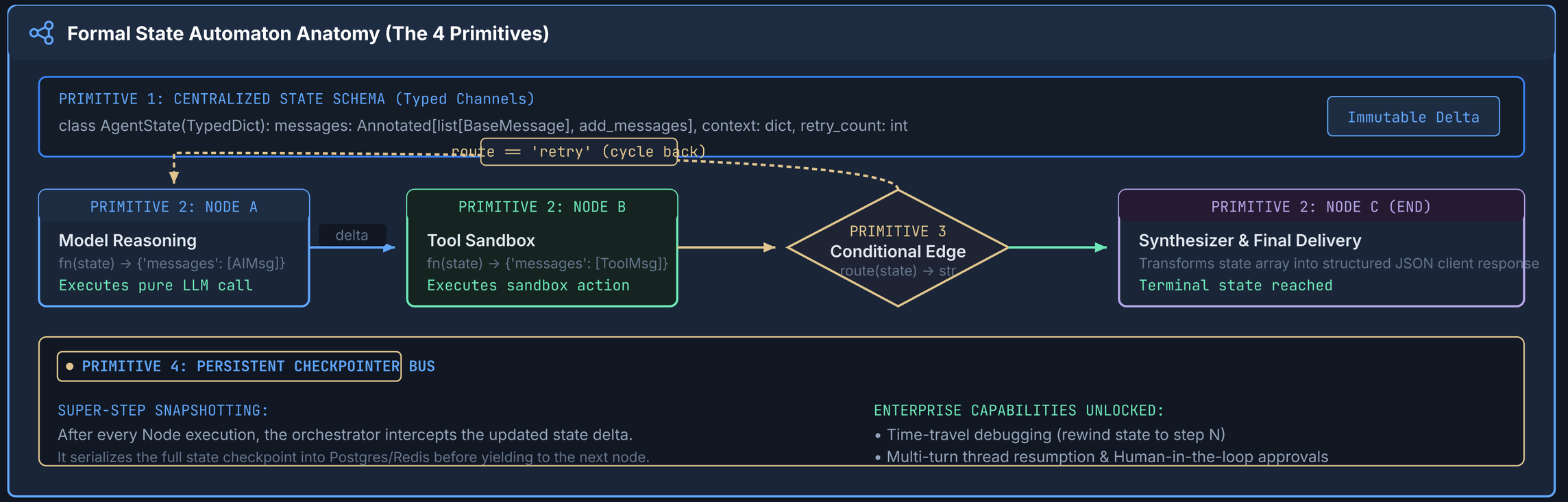
Unbounded sequential LLM inferences compound total user response time linearly per step.
6 tool iterations at 4s per LLM generation plus 1.5s tool latency creates a catastrophic 33-second SLA.

4. THE FIX: GRAPH WRAPPERS WITH HARD GOVERNORS

- Wrap ReAct within a State Graph with explicit recursion_limit = 10.
- Prune tool observations before re-injecting into the message history.
- Route to deterministic fallback handler when loop limits trigger.

The Four Primitives of Graph-Based Orchestration

Abstracting agent execution into State, Nodes, Conditional Edges, and Checkpointers.



State Schemas, Reducers, and Channel Isolation

Defining explicit state schemas with append-only message reducers to eliminate race conditions and state corruption.

TypedDict Schema with Custom Reducers

```python
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages
import operator

class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]
    context: dict[str, Any] # Default: overwrites on update
    audit_log: Annotated[list[str], operator.add] # Appends list
```

REDUCER BEHAVIOR: add_messages

- Appends new HumanMessage, AIMessage, or ToolMessage to state array.
- ID Deduplication: If an incoming message matches an existing message.id, it updates the message in-place rather than duplicating it (crucial for streaming).
- Zero runtime overhead: Native Python typing without Pydantic parsing costs.

Pydantic Domain Validation & Invariants

```python
from pydantic import BaseModel, Field, field_validator

class ReconciliationState(BaseModel):
    invoice_id: str = Field(..., pattern=r'^INV-[0-9]{6}$')
    billed_amount: float = Field(..., gt=0.0)
    approved_by: str | None = None

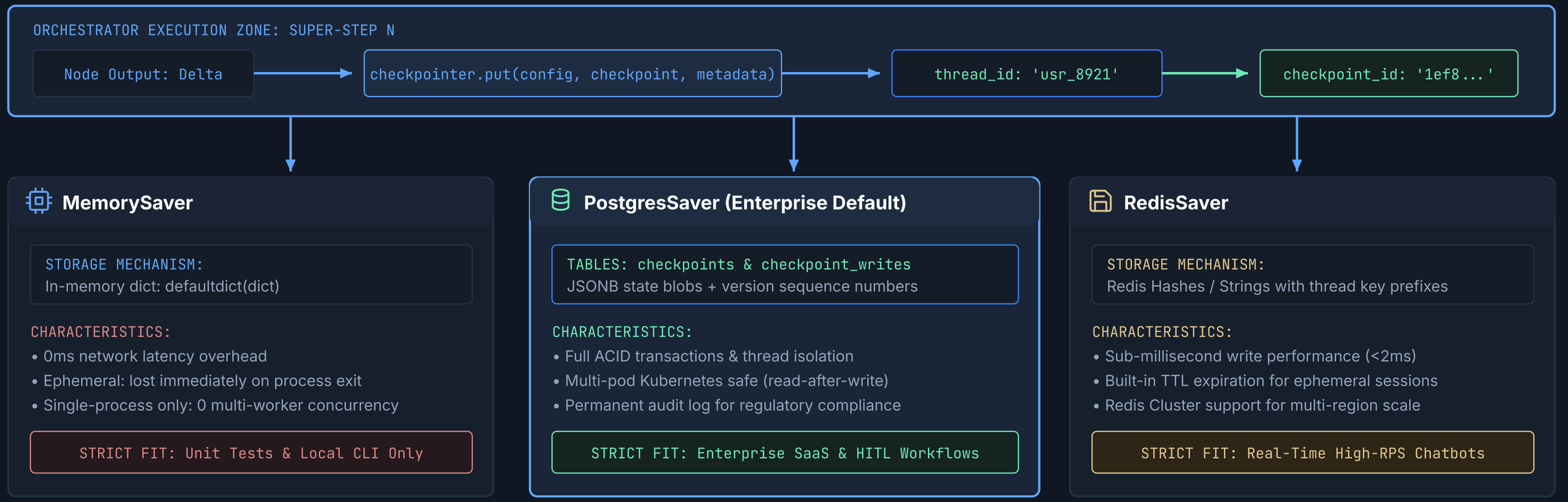
    @field_validator('billed_amount')
    def check_currency(cls, v): return round(v, 2)
```

WHEN TO CHOOSE PYDANTIC OVER TYPEDDICT

- Domain Invariants: Enforces strict business rules before nodes execute.
- Serialization: Built-in .model_dump_json() for clean DB storage.
- Type Coercion: Automatically casts string numbers ('150.00') to float.
- Trade-off: Small CPU overhead (~5-10ms per step) during high-rate validation.

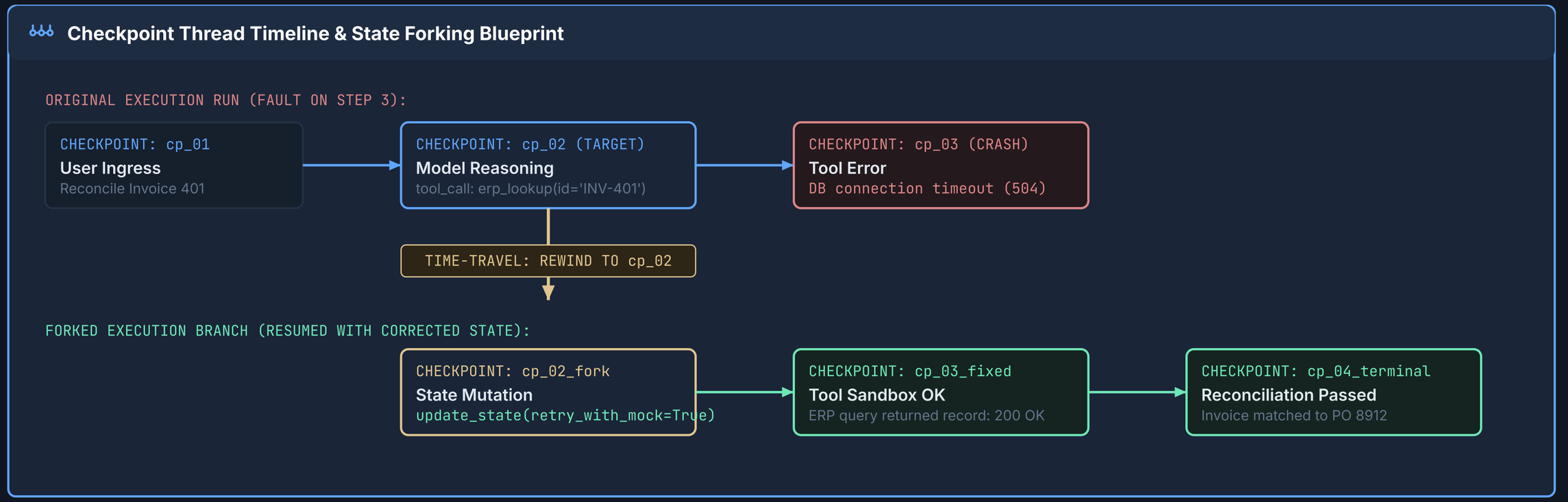
Checkpointer Architecture: Memory, Postgres, and Redis

Checkpointing snapshots agent state at every super-step, enabling fault recovery and multi-turn threads.



Time-Travel Debugging: Replaying and Forking State

Every checkpoint produces an immutable snapshot, allowing developers to rewind, modify, and branch execution.



Tool Calling Contracts: JSON Schemas & Type Coercion

Enforcing strict type boundaries between stochastic model tokens and deterministic business code.

The End-to-End Function Calling Contract Lifecycle

1. TOOL DECLARATION

```
@tool
def get_orders(
 cust_id: str,
 limit: int = 10
) → list[dict]:
```

- Typed Python signature
- Docstrings define intent
- Type hints guide schema

Source of Truth

2. JSON SCHEMA EXPORT

```
{"name": "get_orders",
"parameters": {
 "cust_id": {"type": "str"},
 "limit": {"type": "int"}
}, "required": ["cust_id"]}
```

- Sent to model API request
- Constrains grammar tokens

API Wire Schema

3. MODEL GENERATION

```
tool_calls: [{
 "id": "call_991",
 "name": "get_orders",
 "args": '{"cust_id": "C4"}'
}]
```

- Raw stringified JSON
- Stop sequence triggered
- Stochastic generation

Unvalidated JSON

4. RUNTIME VALIDATION

Pydantic Parser
- Coerce types
- Validate constraints
- Check required fields

- Intercepts malformed JSON
- Emits validation errors
- Prevents code injection

Security Boundary

5. EXECUTION & STATE

Tool Node Execution
- Run Python func
- Sandbox timeout

< 2000ms SLA

- Appends ToolMessage
- Matches call_id
- Updates State channel

State Appended

ENGINEERING CONTRACT INVARIANT:

If validation fails at Stage 4, the runtime must not crash. It formats a ToolMessage(content="ValidationError: ...") and feeds it back to the Model Node to trigger parameter self-correction.

Execution Sandboxing: In-Process vs. Docker vs. MicroVMs

Isolating untrusted model code generation and external tools from the production host environment.

1. In-Process (Dangerous)

EXECUTION ENVIRONMENT:

exec(code) or subprocess.run() in host container

Shares memory, file descriptors, and environment

ISOLATION LEVEL: ZERO (0)

CRITICAL ATTACK VECTORS:

- Env exfiltration: os.environ['DB_PASS']
- Filesystem wipe: shutil.rmtree('/')
- Host resource exhaustion / fork bombs
- SSRF to cloud metadata (169.254.169.254)

PROHIBITED IN PRODUCTION

Acceptable only for pure math tools

2. Container Sandbox (Docker)

EXECUTION ENVIRONMENT:

Ephemeral Docker container with cgroups & namespaces

Volume mounted with read-only root filesystem

ISOLATION LEVEL: OS NAMESPACE

ENGINEERING SPECS:

- Startup latency: 800ms - 2,500ms (container spin)
- Shared Linux host kernel (kernel exploit risk)
- Requires pre-warmed container pools
- Requires --cap-drop=ALL and no-new-privileges

ACCEPTABLE WITH HARDENED CGROUPS

Good for predictable internal batch jobs

3. MicroVMs (E2B / Firecracker)

EXECUTION ENVIRONMENT:

Hardware-virtualized guest kernel (KVM)

Ephemeral guest OS per model execution

ISOLATION LEVEL: HARDWARE VIRTUALIZATION

ENTERPRISE GOLD STANDARD:

- Ultra-fast cold start: ~150ms boot
- Zero host kernel sharing (KVM boundary)
- Strict network egress allowlist by domain
- Automatic teardown upon completion

RECOMMENDED PRODUCTION STANDARD

E2B Sandboxes / AWS Firecracker

TAKEAWAY: Never execute LLM-generated code in the host process; enforce ephemeral microVM isolation with strict network egress controls.

Tool Error Handling: Retries, Reflection, and Circuit Breakers

Designing graceful degradation paths when third-party APIs fail or model parameters hallucinate.

The 4-Tier Tool Fault Tolerance Decision Funnel

TIER 1: TRANSIENT FAULTS

`HTTP 429 / 502 / 503 / 504 Timeout`
Network blips, upstream rate limits

RECOVERY MECHANISM:

- Exponential backoff + Full Jitter

```
t = min(max_t, b * 2^n + rand())
```

- Max 3 retry attempts
- Circuit breaker trips if > 5 faults/min

Action: Retry Tool Node in-place

TIER 2: SCHEMA VALIDATION FAULT

`Pydantic ValidationError`
Missing argument, invalid type/regex

RECOVERY MECHANISM:

- NEVER retry tool code blindly
- Intercept error message & fields
- Append ToolMessage(error=...) to state
- Model Node reflects & adjusts JSON

Action: Cycle to Model Node

TIER 3: PERMANENT / AUTH FAULT

`HTTP 401 / 403 / 404 Entity Missing`
Invalid credentials or forbidden scope

RECOVERY MECHANISM:

- Zero retries (retries waste tokens/cost)
- Deterministic fast-fail path
- Log security audit event to SIEM
- Route directly to Fallback Node

Action: Fast-Fail to Fallback

TIER 4: DEGRADED FALLBACK

`Retries / Budget Exhausted`
Tool completely unreachable

FINAL SAFETY GATE:

- Emit Dead-Letter Queue event
- Deliver partial result to user
- Explanatory error caveat
- Trigger human support ticket

Action: Graceful Delivery

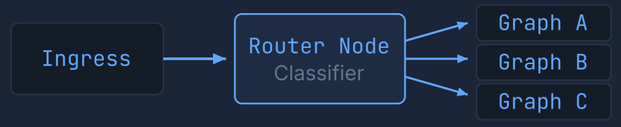
SLA POLICY FOR ENTERPRISE APIS:

Wrap all external calls in an explicit timeout handler (2000ms max). Never let an unhandled tool exception bubble up to terminate the graph.

Taxonomy of Multi-Agent Coordination Topologies

Selecting the optimal network structure based on task modularity, context boundaries, and latency budgets.

PATTERN 1: DETERMINISTIC ROUTER / DISPATCHER



COORDINATION MODEL:

- 1-to-N single-turn dispatch
- Sub-graphs execute in complete isolation
- Minimal token overhead; fast routing

PATTERN 2: SUPERVISOR & WORKER AGENTS



COORDINATION MODEL:

- Central supervisor delegates subtasks
- Workers run in ephemeral scratchpads
- Supervisor synthesizes final result

PATTERN 3: SWARM & PEER-TO-PEER HANDOFFS



COORDINATION MODEL:

- Decentralized direct handoff tools
- Control transferred via transfer_to_agent
- 0 central supervisor latency bottleneck

PATTERN 4: HIERARCHICAL MULTI-TIER TEAMS

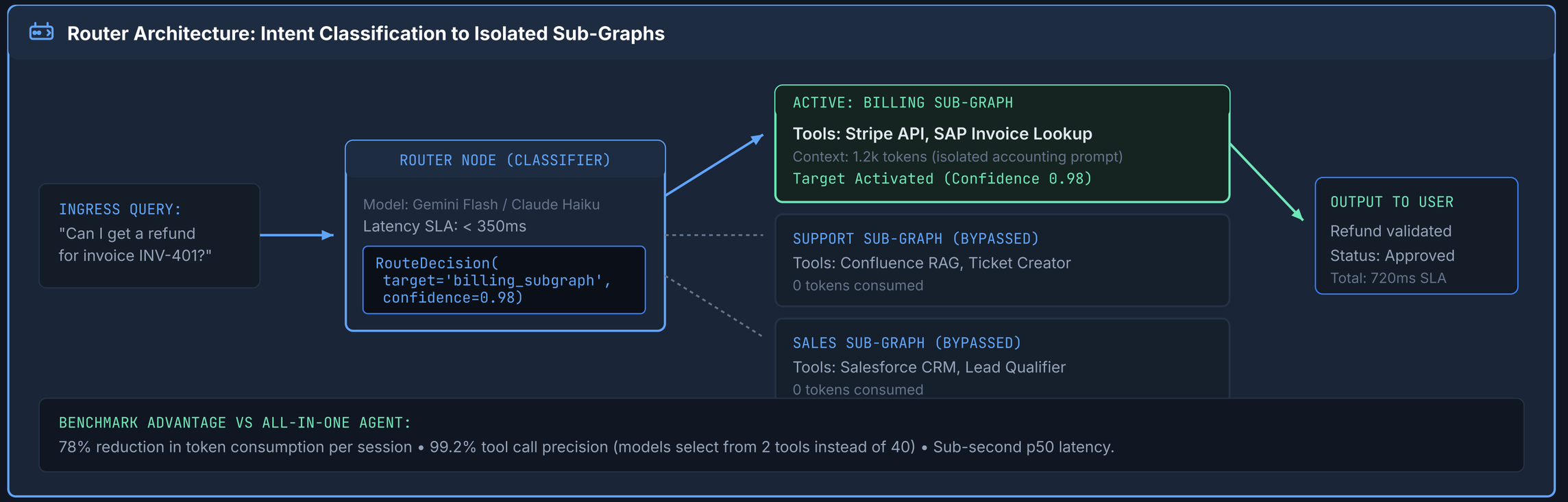


COORDINATION MODEL:

- Multi-tier supervisory management
- Mirrors enterprise organizational structure
- Scales to large software engineering systems

Pattern 1: Router & Dispatcher Architecture

Using a lightweight classifier node to dispatch incoming requests to dedicated, single-purpose sub-graphs.



TAKEAWAY: Routers maximize prompt caching and minimize token overhead by keeping specialized sub-graphs completely decoupled.

Pattern 2: Supervisor & Sub-Agents Architecture

A centralized supervisor maintains master state, delegating specialized subtasks to child worker agents.

Supervisor & Child Workers: Ephemeral Scratchpads & Context Isolation

SUPERVISOR (MASTER ORCHESTRATOR)

Responsibilities:

- Owns the global task decomposition plan
- Delegates subtasks via conditional edges
- Aggregates worker returns into final deliverable

`Current Plan Step: 2 of 3`
Delegate: Coder Agent (Clean Context)

EPHEMERAL CHILD THREAD 1: RESEARCHER AGENT

Executes 4 internal tool calls (Vector RAG, Web Search, PDF parser)
`Returns to Supervisor: 1 paragraph synthesis (180 tokens)`

EPHEMERAL CHILD THREAD 2: CODER AGENT

Generates code in isolated E2B sandbox, runs pytest 3 times
`Returns to Supervisor: Final verified Python file (420 tokens)`

EPHEMERAL CHILD THREAD 3: SECURITY REVIEWER

Audits code for SQL injection, hardcoded secrets, and buffer limits
`Returns to Supervisor: Audit approval token: PASS (45 tokens)`

CONTEXT PROTECTION AUDIT

WITHOUT SUPERVISOR:
45,000 tokens accumulated
(raw bash, pytest, vector chunks)

WITH SUPERVISOR:
Master state: 2,400 tokens
Child threads destroyed upon exit

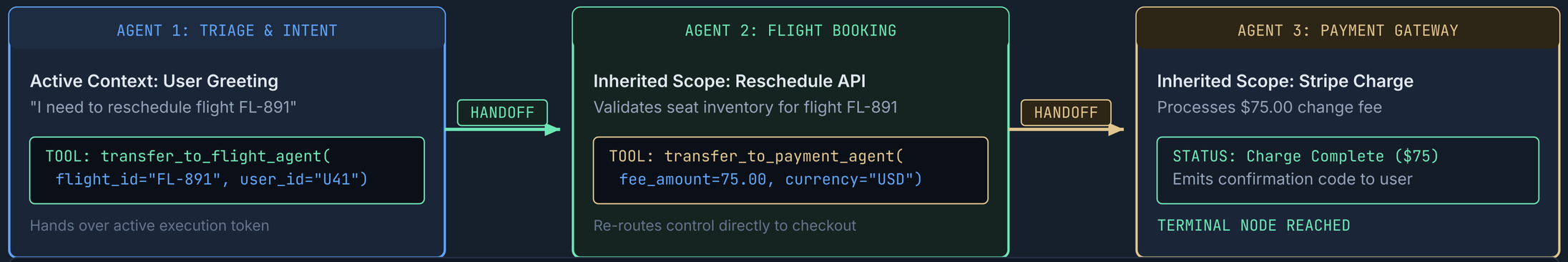
`94.6% Token Reduction`

ARCHITECTURAL RULE:
Child agents must never communicate directly. All state transitions, approvals, and error recoveries flow through the central Supervisor.

Pattern 3: Swarm & Peer-to-Peer Handoff Architecture

Decentralized agents directly transfer execution control and context to peers via handoff functions.

Peer Swarm: Direct Execution Handoff Mechanics



SWARM ENGINEERING TRADE-OFFS

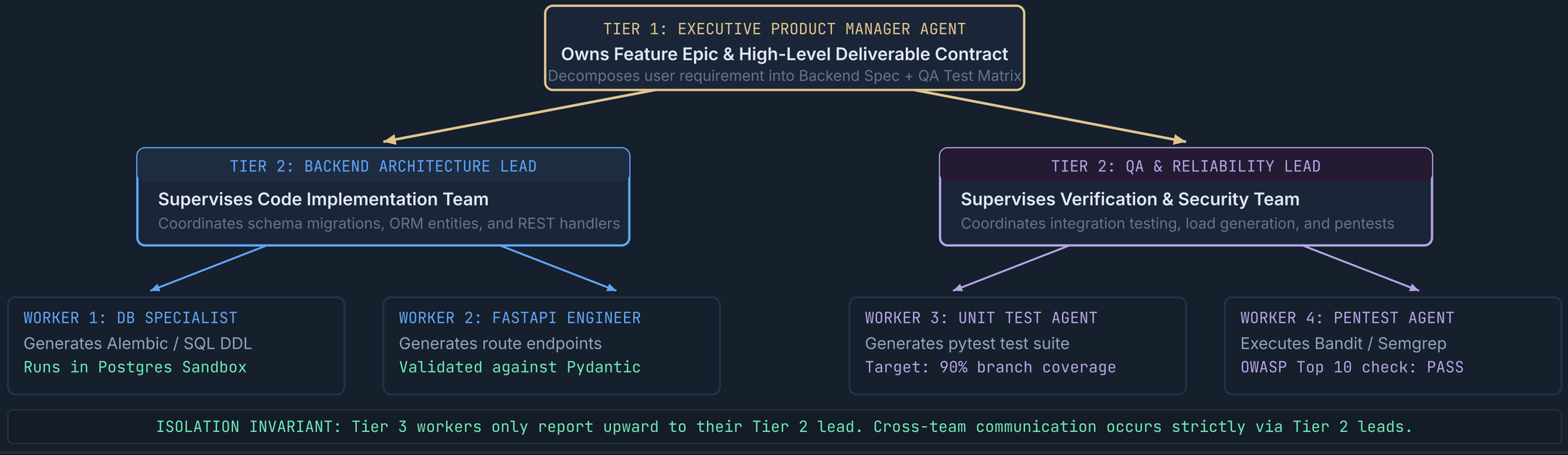
LATENCY ADVANTAGE:
Zero intermediate supervisor hops. Execution transitions immediately in 1 step.

ENTERPRISE RISK (PING-PONG LOOPS):
Agent A can hand off to Agent B, which hands back to Agent A. Runtime MUST enforce max_handoffs = 5.

Pattern 4: Hierarchical Multi-Agent Teams

Scaling agent systems to enterprise software engineering through multi-tier supervisory hierarchies.

Hierarchical Multi-Tier Agent Architecture



TAKEAWAY: Hierarchy mirrors organizational structures: executives manage leads; leads manage specialized contributors.

Context Window Protection: Pruning and Condensation

Preventing quadratic token inflation and prompt degradation across multi-turn agent execution.

Context Window Protection: Trimming, Scrubbing & Condensation

TECHNIQUE 1: MESSAGE TRIMMING

```
trim_messages(
 max_tokens=4000, strategy="last",
 include_system=True, allow_partial=False)
```

HOW IT PROTECTS CONTEXT:

- Retains the foundational SystemMessage.
- Discards oldest intermediate turns once threshold trips.
- Prevents ContextWindowExceededException errors.

TECHNIQUE 2: TOOL PAYLOAD SCRUBBING

```
RAW API: 120kb HTML / 45-field JSON
→ FILTERED: {"status": 200, "balance": 450.00}
```

HOW IT PROTECTS CONTEXT:

- Strips CSS, HTML boilerplate, and metadata keys.
- Retains strictly the typed fields requested by the schema.
- 85% to 95% token savings on tool observation steps.

TECHNIQUE 3: SUMMARY CONDENSATION

```
Background Summarizer Node:
Compresses turns 1..10 into 1 summary message
```

HOW IT PROTECTS CONTEXT:

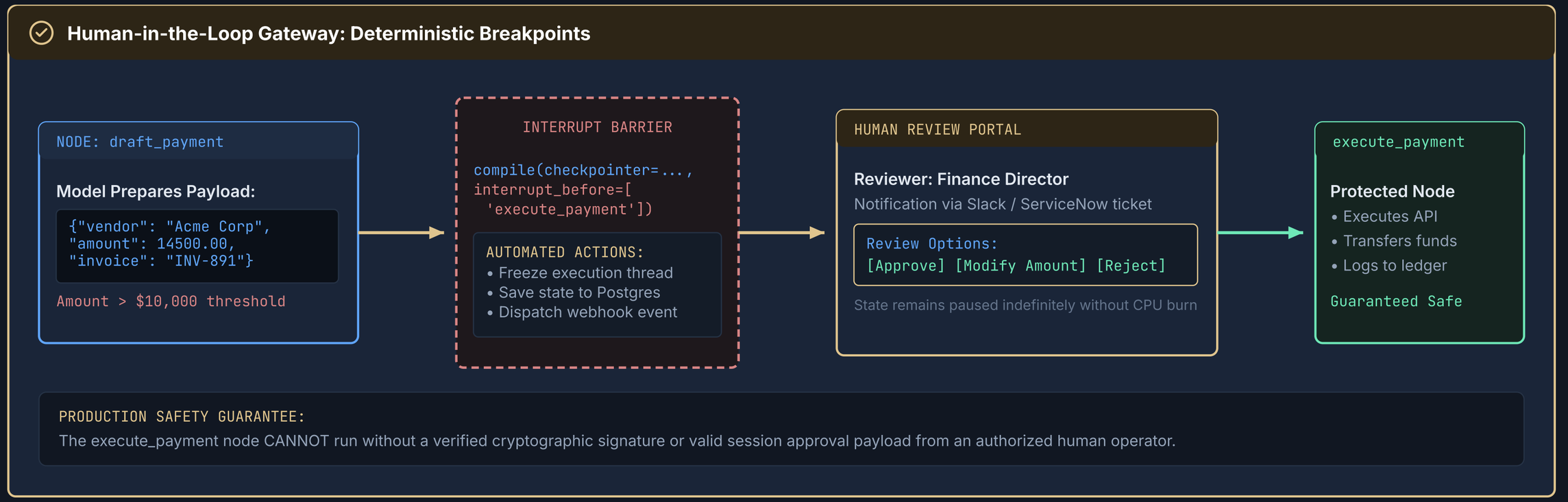
- Preserves critical business facts without token weight.
- Appends summary: HumanMessage(content="[Prior Summary]")
- Keeps token curve flat across 100+ turns.

ENTERPRISE PRODUCTION METRIC (15-TURN WORKFLOW):

Unmanaged thread token usage: 48,500 tokens ($0.48/run) • Managed thread token usage: 3,400 tokens ($0.034/run) • 14.2x Cost Reduction.

Human-in-the-Loop Gateways: Interrupt Architecture

Halting execution before destructive side effects to mandate human authorization and compliance approval.



Resumption Signals: Validating and Mutating Paused State

Resuming paused graphs with human feedback, state overrides, or alternative execution pathways.

State Mutation on Resume: The 3 Approval Patterns

PATTERN A: UNMODIFIED APPROVAL

```
# Resume with None input
app.invoke(
 None, config={"configurable": {"thread_id": "tx_41"}})
```

BEHAVIOR:

- Checkpointer loads frozen state verbatim.
- Execution advances past the interrupt barrier.
- Target node executes with original parameters.

PATTERN B: STATE MUTATION OVERRIDE

```
# 1. Update state prior to resume
app.update_state(config, {
 "amount": 12000.00, "override_by": "alice"})
app.invoke(None, config) # 2. Resume
```

BEHAVIOR:

- Overwrites target state fields in checkpointer.
- Intercepts model hallucination before side-effect.
- Node executes with human-sanitized value.

PATTERN C: REJECTION & ABORT

```
# Inject rejection status
app.update_state(config, {
 "status": "REJECTED_BY_CFO"})
```

BEHAVIOR:

- Graph conditional edge routes to cancel_node.
- Destructive node is completely bypassed.
- Audit entry logged to security event stream.

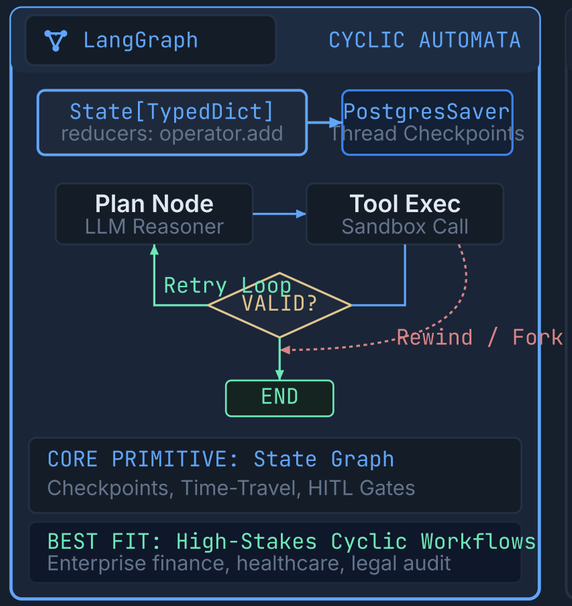
COMPLIANCE TRAIL IN PERSISTENT STORAGE:

Every state mutation creates a new sequential checkpoint. Auditors can inspect both what the LLM initially drafted and what the human reviewer edited before execution.

Enterprise Agent Frameworks: Four Architectural Archetypes

Visualizing the underlying architectural engines: state graphs, event buses, group chat loops, and enterprise kernels.

Four Distinct Orchestration Engines: Architectural Blueprints



CORE PRIMITIVE: State Graph
Checkpoints, Time-Travel, HITL Gates

BEST FIT: High-Stakes Cyclic Workflows
Enterprise finance, healthcare, legal audit



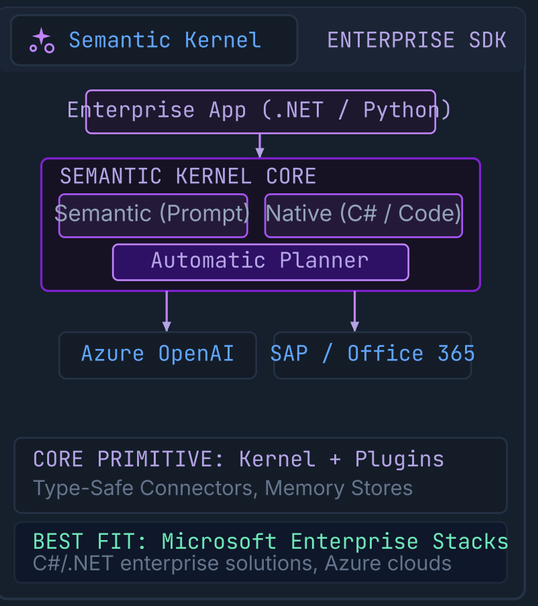
CORE PRIMITIVE: Event Pub/Sub
Asyncio Queue, Decoupled Steps

BEST FIT: Advanced RAG Pipelines
Knowledge graphs, doc parsers, ETL



CORE PRIMITIVE: Conversational Personas
Backstories, Tasks, Peer Message Exchange

BEST FIT: Simulation & Fast PoCs
Role-play benchmarks, rapid ideation



CORE PRIMITIVE: Kernel + Plugins
Type-Safe Connectors, Memory Stores

BEST FIT: Microsoft Enterprise Stacks
C#/.NET enterprise solutions, Azure clouds

Framework Comparison Matrix: Technical Dimensions

Rigorous comparative analysis across state persistence, graph cyclicity, typing, and production readiness.

Enterprise Architectural Comparison Matrix

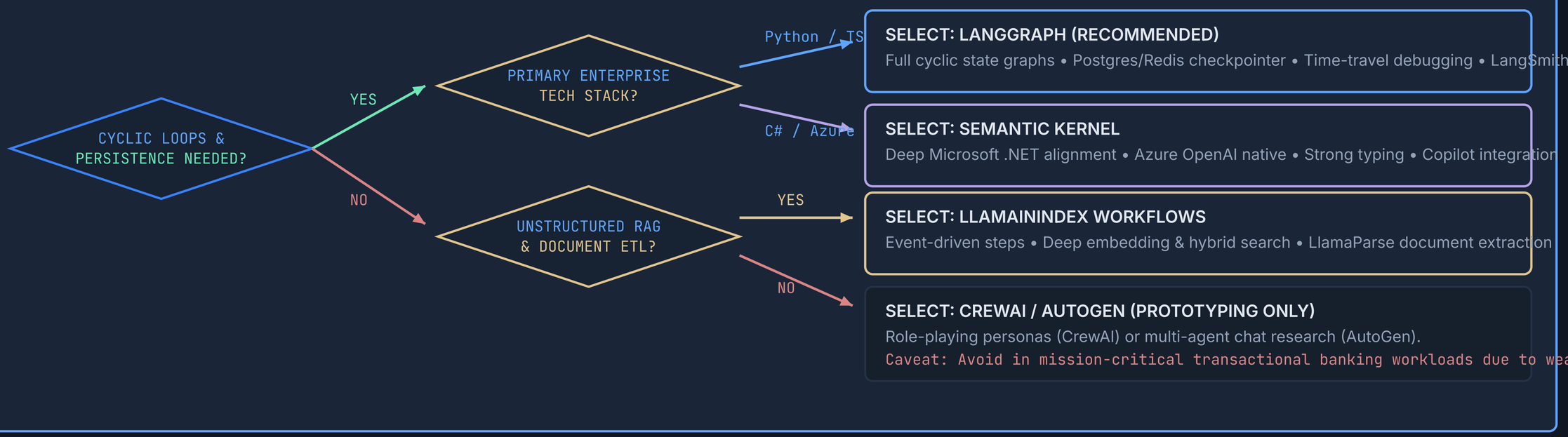
FRAMEWORK	TOPOLOGY	STATE PERSISTENCE	TYPE SAFETY	HITL MECHANICS	PROD MATURITY
LangGraph	Cyclic State Graph	Postgres / Redis / Memory	TypedDict + Pydantic	interrupt_before / mutate	High (Tier 1)
LlamaIndex Workflows	Async Event Pub/Sub	LlamaCloud / Key-Value	Pydantic Event Classes	Event Listener Pauses	High (Data/RAG)
AutoGen (MSFT)	Conversational Chat Loop	Disk Cache / Custom JSON	Dict / Dynamic Messages	HumanInputMode.ALWAYS	Moderate (Research)
CrewAI	Sequential / Hierarchical	SQLite / Memory Store	Pydantic Task Outputs	CLI Human Input Flag	Early (PoC Fast)
Semantic Kernel	Plan-Execute Pipeline	Azure Cosmos / Redis	Strict C# / Pydantic	Filter Middleware Hooks	High (MSFT .NET)

TAKEAWAY: For complex cyclic logic and time-travel debugging, LangGraph leads; for enterprise Microsoft environments, Semantic Kernel leads.

Architectural Selection Tree: Framework Choice

Guiding enterprise architecture teams to the defensible framework based on workflow constraints.

Enterprise Architecture Selection Tree



Distributed Tracing: OpenTelemetry, LangSmith, and Phoenix

Instrumenting multi-step agent runs with span hierarchies, latency waterfalls, and token accounting.

Graph-Aware Distributed Trace Waterfall & Telemetry Bus



OTEL GENAI SEMANTIC CONVENTIONS

```
gen_ai.system: "langgraph"
gen_ai.request.model: "claude-3-5-sonnet"
gen_ai.usage.input_tokens: 10850
gen_ai.usage.output_tokens: 1550
gen_ai.response.finish_reasons: ["tool_calls"]
thread.id: "usr_reconcile_8921"
checkpoint.id: "1ef892a0b1"
```

SUPPORTED OBSERVABILITY BACKENDS:

LangSmith | OpenTelemetry | Arize Phoenix

Export spans over gRPC/OTLP to Datadog, Honeycomb, or Dynatrace.

Loop Detection, Recursion Limits, and Cost Governors

Implementing circuit breakers to terminate runaway agent oscillations and enforce hard cost ceilings.

⚠ Defense-in-Depth: Runaway Loop Prevention & Cost Circuit Breakers

LAYER 1: RECURSION DEPTH CAP

```
app.invoke(inputs,
 config={"recursion_limit": 25})
```

HOW IT GOVERNS RUNS:

- Hard limit on total super-step node transitions.
- Throws GraphRecursionError on step 26.
- Prevents unbounded graph traversals.

LAYER 2: STATE HASH CYCLE DETECTOR

```
h = md5(last_3_tool_signatures)
if seen_hashes[h] > 2: trip_circuit()
```

HOW IT DETECTS OSCILLATION:

- Detects identical tool calls repeated consecutively.
- Catches A → B → A → B ping-pong oscillations.
- Interrupts before token budget is burned.

LAYER 3: FINANCIAL BUDGET BREAKER

```
if total_cost_usd > 0.50 or
total_tokens > 40000: abort_run()
```

HOW IT ENFORCES BUDGETS:

- Real-time token counter updated on every LLM span.
- Enforces strict corporate cost ceilings ($0.50 max/run).
- Routes to graceful degraded fallback on trip.

ACTION ON GOVERNOR TRIP:

Graph pauses immediately • State status updated to TIMEOUT_CIRCUIT_BROKEN • PagerDuty alert emitted • Safe fallback response delivered to end-user.

Trajectory Evaluation: Auditing Intermediate Reasoning

Why evaluating end-to-end answers fails in agents; verifying tool selection, arguments, and intermediate states.

Trajectory Evaluation: Intermediate Step Auditing Framework

SUPERFICIAL: FINAL ANSWER ONLY (FLAWED)

```
Input: "Find invoice total for Acme Corp"
Final Output: "$14,500.00" (Matches Ground Truth!)
```

HIDDEN ARCHITECTURAL DEFECTS:

- Step 1: Model called web_search (unauthorized tool leak)
- Step 2: Failed SQL query with syntax error (wasted tokens)
- Step 3: Hardcoded invoice total from memory (hallucination)
- Total latency: 14.8 seconds (SLA breach)

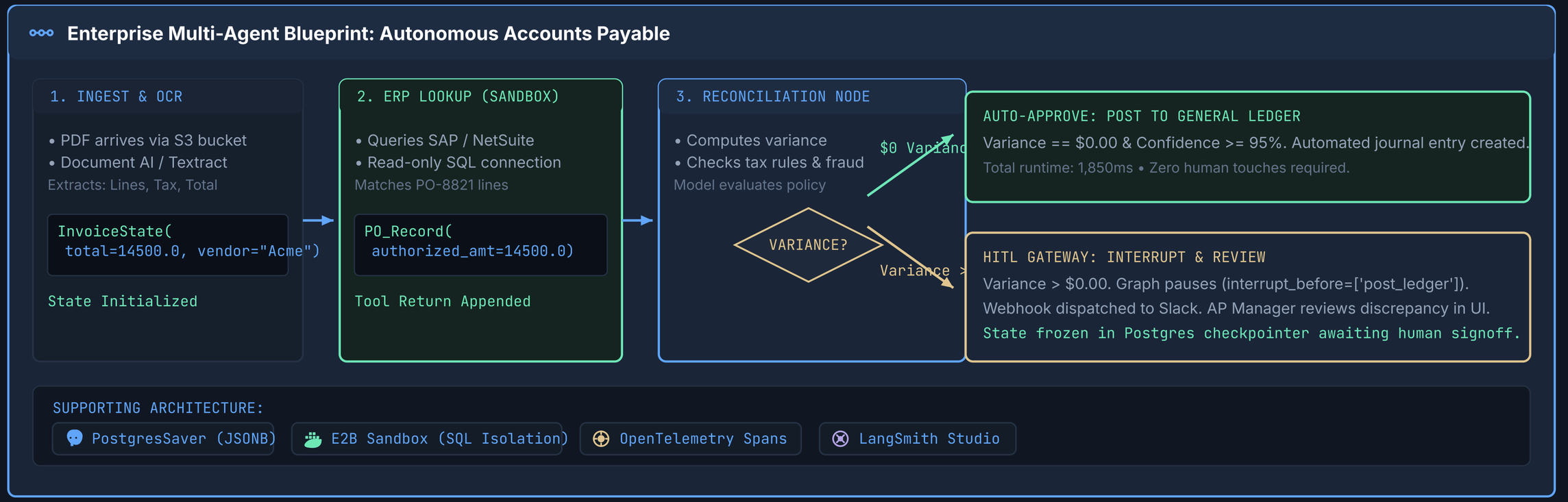
VERDICT: FALSE POSITIVE (RISK HIGH)

ENTERPRISE STANDARD: STEP-BY-STEP TRAJECTORY SCRUTINY

1. TOOL SELECTION ACCURACY (SCORE: 1.0)
   Did the model invoke erp_lookup rather than web_search or arbitrary SQL?
2. ARGUMENT INTEGRITY (SCORE: 1.0)
   Were parameters formatted to exact schema regex (vendor_id='V-891', date='2026-09-01')?
3. INTERMEDIATE GROUNDING (SCORE: 1.0)
   Did the agent's intermediate synthesis match tool return without hallucinating numbers?
4. TRAJECTORY EFFICIENCY (SCORE: 0.95)
   Did it reach the goal in the minimal 3 super-steps without redundant polling loops?

Production Case Study: Autonomous Financial Reconciliation

A resilient multi-agent graph with OCR extraction, ERP integration, anomaly checks, and HITL authorization.



Live Demo Tour: LangGraph Studio & State Inspection

A safe, 10-15 minute visual walkthrough of graph compilation, breakpoint hits, and state mutation.

Instructor Console Walkthrough: 5-Stage Demo Map

STAGE 1: COMPILE

Graph Compilation

- Open LangGraph Studio
- Point to workflow graph
- Show visual canvas

```
app = builder.compile(
 checkpointer=saver)
```

Duration: 2-3 minutes

STAGE 2: INJECT

Inject Test Payload

- Paste invoice JSON
- Click 'Start Run'
- Watch nodes illuminate

```
State: Step 1 (OCR)
→ Step 2 (ERP Match)
```

Duration: 2-3 minutes

STAGE 3: BREAKPOINT

Inspect Paused State

- Graph halts at barrier
- Open State Inspector
- Show variance: $450.00

```
STATUS: INTERRUPTED
Awaiting human review
```

Duration: 3 minutes

STAGE 4: MUTATE

Live State Mutation

- Edit state variable in UI
- Set approved_by="cfo"
- Click 'Resume Graph'

```
State Updated: 200 OK
Graph resumes to END
```

Duration: 3 minutes

STAGE 5: TRACE

Audit Tracing Spans

- Open LangSmith trace
- Inspect span latency
- Review token spend

```
Total Tokens: 3,420
Run Cost: $0.012
```

Duration: 3 minutes

INSTRUCTOR PREPARATION CHECKLIST:

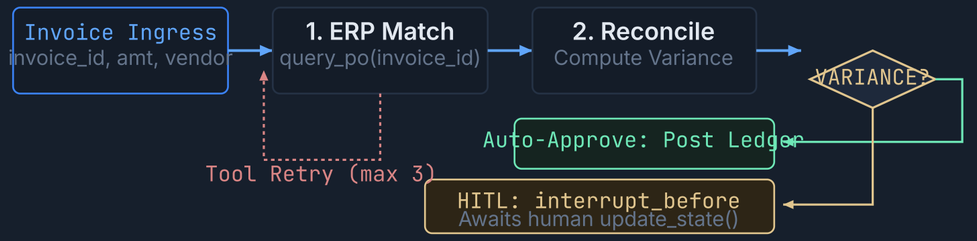
Run locally with MemorySaver for zero cloud dependency • Pre-load sample invoice JSON payload in clipboard • Keep LangGraph Studio window open in Chrome tab 2.

Capstone Lab: Resilient AP Invoice & Dispute Resolution Agent

Build or specify an end-to-end cyclic state machine with SQLite checkpointer, tool circuit breaker, and HITL authorization.

Capstone Engineering Specification: Autonomous Accounts Payable Agent

REQUIRED STATE MACHINE TOPOLOGY



PERSISTENCE CONTRACT: SqliteSaver / PostgresSaver

- State Schema: TypedDict with messages (add_messages), invoice, variance, retries.
- Checkpoints written on every superstep. Resumption verified via thread_id.

CHOOSE ONE TRACK (100% ZERO-COST GUARANTEED)

OPTION A: ARCHITECTURE SPEC

- Target: Architects & Technical Leads
- Format: Markdown document + Mermaid

Key Requirements:

1. Mermaid state graph diagram
2. Full TypedDict state schema
3. Tool contract & error matrix
4. HITL sequence & payload specs
5. OTel span hierarchy plan

Requires zero code; pure engineering rigor

OPTION B: PYTHON PROTOTYPE

- Target: Software & AI Engineers
- Format: Python repo (LangGraph)

Key Requirements:

1. state.py with TypedDict schema
2. tools.py with mock SQLite DB
3. graph.py with SqliteSaver
4. interrupt_before and resume test
5. Automated pytest suite passing

Runs on laptop with Ollama or mock ($0)

CAPSTONE EVALUATION RUBRIC (100 POINTS TOTAL):

- State Schema & Reducers: 25 pts
- Cyclic Recovery & Circuit Breaker: 25 pts
- Tool Sandbox & Schemas: 20 pts
- HITL Breakpoint & Mutate: 15 pts
- Telemetry & Tests: 15 pts